

Deep Learning Fondamentaux

Entraînement, Optimisation et Stabilisation

Plan du Cours

- 1 Rappels & Pourquoi le Deep Learning ?
- 2 L'Entraînement et ses Limites
- 3 Fonctions d'Activation
- 4 Initialisation des Poids
- 5 L'Optimisation Avancée
- 6 Régularisation : Lutter contre l'Overfitting
- 7 Stabilisation : Batch Normalization
- 8 Conclusion

Rappels & Pourquoi le Deep Learning ?

Le Perceptron

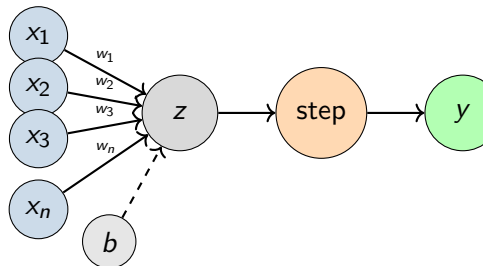
Architecture :

- Entrées : $x_1, x_2, \dots, x_n$
- Poids : $w_1, w_2, \dots, w_n$
- Biais : b
- Sortie : $y \in \{0, 1\}$

Formule mathématique :

$$z = \sum_{i=1}^n w_i x_i + b$$

$$y = \text{step}(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases}$$

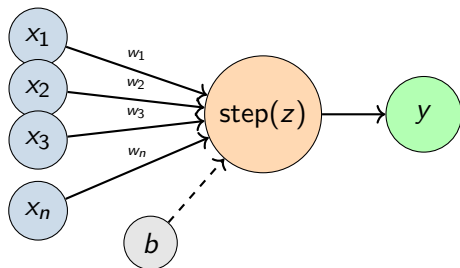


Le Perceptron

Formule mathématique :

$$y = \text{step} \left(\sum_{i=1}^n w_i x_i + b \right)$$

Les deux opérations sont combinées dans un seul nœud.



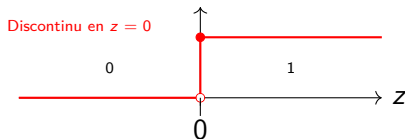
Convention

Dans les diagrammes de réseaux de neurones, on représente la **somme pondérée** et la **fonction d'activation** comme un seul nœud pour simplifier la visualisation.

Limitation de la fonction d'activation Step

- La fonction **step** n'est **pas dérivable** en $z = 0$.
- Impossible de calculer un gradient.
- Donc impossible d'utiliser la **descente de gradient**.

Graphique de la fonction Step
 $\text{step}(z)$



Solution

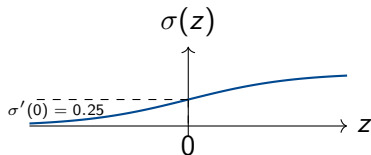
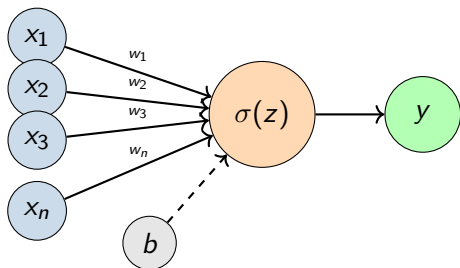
Remplacer **step** par une fonction **dérivable** : la **sigmoïde** !

Formule mathématique :

$$z = \sum_{i=1}^n w_i x_i + b$$

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$



Fonction d'activation : Sigmoid

Avantage de la sigmoïde

Avec la sigmoïde, on peut utiliser la **descente de gradient** !

Pourquoi ça marche ?

- $\sigma(z)$ est **continue et dérivable** partout.
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ existe toujours.
- On peut calculer le gradient en chaque point.

Conséquence :

- On peut calculer comment chaque poids influence l'erreur.
- On peut ajuster les poids pour réduire l'erreur.
- Le réseau peut **apprendre** automatiquement !

Comment faire la Descente de Gradient ?

Objectif : Calculer $\frac{\partial \mathcal{L}}{\partial w_i}$ pour ajuster les poids.

Le Problème

Notre fonction est une **composition** :

$$\mathcal{L}(y, \sigma(z)) \quad \text{où } \mathcal{L} \text{ est la fonction de perte et } z = \sum_{i=1}^n w_i x_i + b$$

Comment calculer la dérivée d'une composition ?

La Règle de la Chaîne (Chain Rule)

Pour une composition $f(g(x))$, la dérivée est :

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg} \cdot \frac{dg}{dx}$$

On multiplie les dérivées le long de la chaîne.

Application de la Règle de la Chaîne

Exemple concret : Calculer $\frac{\partial \mathcal{L}}{\partial w_i}$

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \sigma} \cdot \frac{\partial \sigma}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

Étape par étape :

- ① $\frac{\partial \mathcal{L}}{\partial \sigma}$: Comment l'erreur change avec la sortie
- ② $\frac{\partial \sigma}{\partial z}$: Comment la sigmoïde change avec z
- ③ $\frac{\partial z}{\partial w_i}$: Comment z change avec le poids w_i

Calculs :

- $\frac{\partial z}{\partial w_i} = x_i$ (simple !)
- $\frac{\partial \sigma}{\partial z} = \sigma(z)(1 - \sigma(z))$
- $\frac{\partial \mathcal{L}}{\partial \sigma}$ dépend de la fonction de perte

Résultat

On peut maintenant ajuster : $w_i \leftarrow w_i - \alpha \frac{\partial \mathcal{L}}{\partial w_i}$

Le Problème : Linéarité

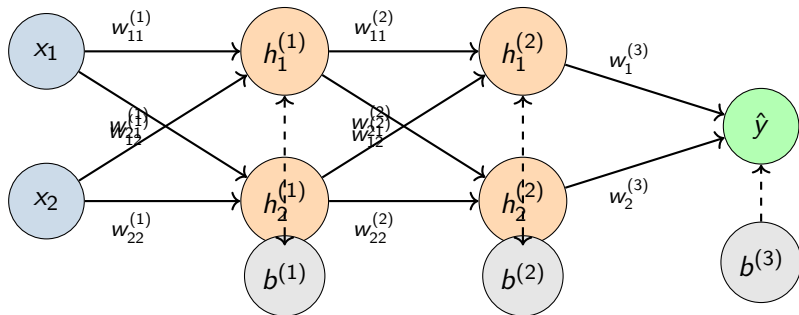
Le perceptron et la régression logistique sont des **classificateurs linéaires**.

- Ils ne peuvent apprendre que des **frontières de décision linéaires**.
- Exemple : Impossible de résoudre le problème **XOR**.

Solution

Il faut **empiler plusieurs couches** pour apprendre des représentations non-linéaires !

Le MLP (Multilayer Perceptron) : Architecture



Le MLP (Multilayer Perceptron) : Formules Mathématiques

Forward Pass :

$$h_1^{(1)} = \sigma \left(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + b_1^{(1)} \right)$$

$$h_2^{(1)} = \sigma \left(w_{12}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} \right)$$

$$h_1^{(2)} = \sigma \left(w_{11}^{(2)} h_1^{(1)} + w_{21}^{(2)} h_2^{(1)} + b_1^{(2)} \right)$$

$$h_2^{(2)} = \sigma \left(w_{12}^{(2)} h_1^{(1)} + w_{22}^{(2)} h_2^{(1)} + b_2^{(2)} \right)$$

$$\hat{y} = \sigma \left(w_1^{(3)} h_1^{(2)} + w_2^{(3)} h_2^{(2)} + b^{(3)} \right)$$

Chaque neurone calcule une somme pondérée suivie de l'activation sigmoïde.

L'Entraînement et ses Limites

Le Cycle d'Entraînement

- ➊ **Forward Pass** : On passe la donnée dans le réseau pour obtenir une prédiction $\hat{y}$.
- ➋ **Loss Function** : On calcule l'erreur $\mathcal{L}(y, \hat{y})$.
- ➌ **Backward Pass (Rétro-propagation)** : On calcule les gradients $\frac{\partial \mathcal{L}}{\partial w}$ pour tous les poids.
- ➍ **Gradient Descent** : On ajuste les poids : $w \leftarrow w - \alpha \frac{\partial \mathcal{L}}{\partial w}$

Règle de la Chaîne (encore)

Comme pour la régression logistique, on utilise la **règle de la chaîne**.

Exemple : Calculer $\frac{\partial \mathcal{L}}{\partial w_{1,1}^{(1)}}$

Notation : $w_{1,1}^{(1)}$ = poids de x_1 vers $h_1^{(1)}$ (premier input, premier neurone)
Pour un MLP, on a une composition de fonctions :

$$\mathcal{L}(y, \hat{y}) \quad \text{où} \quad \hat{y} = \sigma(W^{(3)}h^{(2)} + b^{(3)})$$

$$h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)}) \quad h^{(1)} = \sigma(W^{(1)}x + b^{(1)})$$

Le gradient remonte la chaîne :

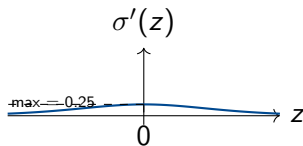
$$\frac{\partial \mathcal{L}}{\partial w_{1,1}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h^{(2)}} \cdot \frac{\partial h^{(2)}}{\partial h^{(1)}} \cdot \frac{\partial h^{(1)}}{\partial w_{1,1}^{(1)}}$$

Le Problème avec la Sigmoid

Rappel : La dérivée de la sigmoïde est $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Propriété importante :

- $\sigma'(z) \leq 0.25$ (maximum en $z = 0$)
- Pour $|z|$ grand, $\sigma'(z) \approx 0$
- La sigmoïde **sature** facilement.



La Disparition du Gradient

Que se passe-t-il dans un réseau profond ?

Exemple avec 3 couches

Pour calculer $\frac{\partial \mathcal{L}}{\partial w^{(1)}}$, on multiplie :

$$\frac{\partial \mathcal{L}}{\partial w^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \underbrace{\sigma'(z^{(3)})}_{\leq 0.25} \cdot \underbrace{\sigma'(z^{(2)})}_{\leq 0.25} \cdot \underbrace{\sigma'(z^{(1)})}_{\leq 0.25}$$

Si chaque $\sigma' \approx 0.25$, alors : $0.25 \times 0.25 \times 0.25 = 0.0156$ **Avec 10**

couches : $0.25^{10} \approx 0.0000009 \rightarrow$ Le gradient a **disparu** !

Conséquence

Les premières couches n'apprennent plus rien. Le signal d'erreur meurt avant d'atteindre le début du réseau.

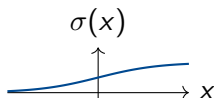
Fonctions d'Activation

Le premier correctif immédiat pour la disparition du gradient.

Sigmoïde & Tanh

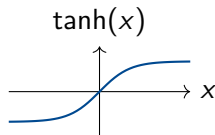
Sigmoïde : $\sigma(x) = \frac{1}{1+e^{-x}}$

- Sortie : $[0, 1]$
- Dérivée max : $\sigma'(0) = 0.25$
- **Problème** : Sature rapidement



Tanh : $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

- Sortie : $[-1, 1]$ (centrée)
- Dérivée max : $\tanh'(0) = 1$
- **Problème** : Sature toujours !



Problème Commun

Les deux saturent : pour $|x| > 2$, la dérivée $\approx 0 \rightarrow$ gradient disparaît !

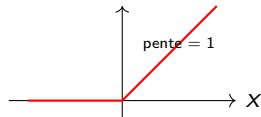
ReLU : $\text{ReLU}(x) = \max(0, x)$

Dérivée :

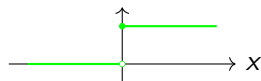
$$\text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

ReLU et sa dérivée

$\text{ReLU}(x)$



$\text{ReLU}'(x)$



Pourquoi ça marche ?

- Pour $x > 0$: dérivée = **1** (pas de saturation !)
- $1 \times 1 \times 1 \times \dots = 1 \rightarrow$ gradient intact
- Calcul très rapide (pas d'exponentielle)

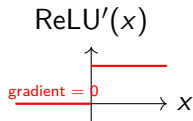
Victoire

Avec ReLU, le gradient peut traverser de nombreuses couches sans disparaître !

Le Problème de ReLU : "Dead ReLU"

Le Problème :

- Si $x < 0$: sortie = 0, dérivée = 0
- **Neurone Mort** : n'apprend plus jamais
- Gradient bloqué $\rightarrow$ pas de mise à jour



Solution : Variantes

- **Leaky ReLU** :

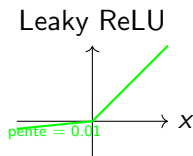
$$f(x) = \max(0.01x, x)$$

- **PReLU** : $f(x) = \max(\alpha x, x)$ où α est appris

- **ELU** :

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha(e^x - 1) & \text{si } x \leq 0 \end{cases}$$

(plus lisse mais coûteux)



Résumé : Quelle Activation Utiliser ?

Couches Cachées :

- Par défaut : **ReLU**
- Neurones morts ? → **Leaky ReLU**
- Besoin de lissage ? → **ELU**

Couche de Sortie :

- Classification Binaire → **Sigmoïde**
- Classification Multi-classe → **Softmax**
- Régression → **Linéaire** (pas d'activation)

Activation	Dérivée Max	Usage
Sigmoïde	0.25	Sortie (binaire)
Tanh	1.0	Rarement utilisé
ReLU	1.0	Cachées (standard)
Leaky ReLU	1.0	Cachées (si dead ReLU)

Initialisation des Poids

Pourquoi "Random" n'est pas suffisant.

Pourquoi l'Initialisation est Cruciale ?

Le Problème

On ne peut pas apprendre sans une bonne base. Les poids initiaux déterminent si le réseau peut apprendre efficacement.

Problème 1 : Initialisation à Zéro

- Tous les poids = 0
- Tous les neurones calculent la même chose
- Ils reçoivent le même gradient

Conséquence

Les neurones restent identiques → ils n'apprennent jamais de caractéristiques différentes !

Solution

Initialiser avec des valeurs aléatoires (petites) pour briser la symétrie.

Problème 2 : Poids Trop Grands

- Poids initiaux très grands (ex : $w = 10$)
- Les activations deviennent très grandes
- La sigmoïde sature : $\sigma(10) \approx 1$
- Dérivée $\approx 0 \rightarrow$ gradient disparaît

Exemple

$$z = 10x_1 + 10x_2 + b$$

Si $x_1, x_2 > 0$: z très grand

$$\sigma(z) \approx 1 \text{ (saturé)}$$

$$\sigma'(z) \approx 0 \text{ (pas de gradient)}$$

Conséquence

Gradient explose ou disparaît $\rightarrow$ le réseau ne peut pas apprendre.

Problème 3 : Poids Trop Petits

- Poids initiaux très petits (ex : $w = 0.001$)
- Les activations deviennent très petites
- Le signal s'affaiblit à chaque couche
- Après plusieurs couches : signal ≈ 0

Exemple

Couche 1 : $h^{(1)} = \sigma(0.001 \cdot x) \approx 0.5$

Couche 2 : $h^{(2)} = \sigma(0.001 \cdot h^{(1)}) \approx 0.5$

Couche 10 : signal presque nul

Conséquence

Le gradient disparaît en remontant $\rightarrow$ les premières couches n'apprennent rien.

Problème de l'Initialisation

Initialisation	Problème	Conséquence
Zéro	Symétrie	Neurones identiques
Trop grands	Saturation	Gradient disparaît
Trop petits	Signal faible	Gradient disparaît
Optimal	Variance adaptée	Apprentissage possible

Solution

Il faut initialiser avec une **variance adaptée** à la taille de la couche et à la fonction d'activation utilisée.

Principe

Garder la **variance des signaux constante** à travers les couches.

Formule :

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

ou uniforme :

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Notation :

- n_{in} = nombre de neurones en **entrée** de la couche
- n_{out} = nombre de neurones en **sortie** de la couche

Usage :

- Sigmoidé, Tanh
- Fonctions saturantes

Couche 784 $\rightarrow$ 256

- $n_{\text{in}} = 784$ (neurones en entrée)
- $n_{\text{out}} = 256$ (neurones en sortie)

Calcul de la variance :

$$\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}} = \frac{2}{784 + 256} = \frac{2}{1040} \approx 0.0019$$

Initialisation :

$$W \sim \mathcal{N}(0, 0.0019)$$

ou uniforme :

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{1040}}, \sqrt{\frac{6}{1040}}\right) \approx \mathcal{U}(-0.076, 0.076)$$

Pourquoi garder la variance constante ?

- **Évite l'explosion** : Si variance augmente $\rightarrow$ signaux deviennent trop grands $\rightarrow$ saturation
- **Évite la disparition** : Si variance diminue $\rightarrow$ signaux deviennent trop petits $\rightarrow$ gradient disparaît
- **Apprentissage stable** : Toutes les couches reçoivent des signaux dans la même plage
- **Convergence rapide** : Pas besoin d'ajuster les learning rates par couche

Résultat

Chaque couche reçoit des signaux avec une variance similaire, permettant un apprentissage uniforme à travers tout le réseau.

Problème avec ReLU

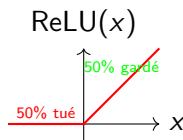
ReLU "tue" la moitié de la distribution (pour $x < 0$) $\rightarrow$ variance divisée par 2.

Solution : Multiplier la variance de Xavier par 2 :

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

ou uniforme :

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right)$$



Correction : Variance $\times 2$ pour compenser

Règle d'Or

Toujours utiliser **He Initialization** avec **ReLU**.

L'Optimisation Avancée

Comment descendre la montagne plus vite et sans tomber ?

Gradient Descent Classique : Ce qu'on Fait

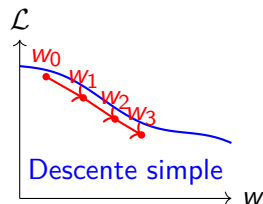
L'algorithme de base

Gradient Descent simple : $w_{t+1} = w_t - \alpha \nabla \mathcal{L}(w_t)$

À chaque itération

- 1 Calculer le gradient : $\nabla \mathcal{L}(w_t)$
- 2 Mettre à jour :
 $w_{t+1} = w_t - \alpha \nabla \mathcal{L}(w_t)$

Visualisation :



Gradient Descent Classique : Les Limites

Problèmes rencontrés :

① Learning rate fixe :

- Trop petit $\rightarrow$ convergence lente
- Trop grand $\rightarrow$ oscillations ou divergence

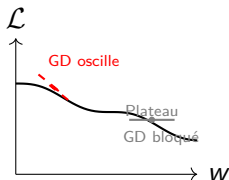
② Pas de mémoire :

- Oublie les gradients précédents
- "Hésite" dans les plateaux

③ Traitement uniforme :

- Même learning rate pour tous
- Certains oscillent, d'autres stagnent

Visualisation :



Le Paysage d'Erreur

Pourquoi c'est difficile ?

Le paysage d'erreur des réseaux profonds n'est pas une simple cuvette convexe.

Problèmes du paysage :

- **Points de selle** : $\nabla = 0$ mais pas un minimum
- **Minima locaux** : Moins problématiques qu'on pense
- **Plateaux** : Zones plates où $\nabla \approx 0$
- **Ravins étroits** : Directions avec gradients très différents

Conséquence

La descente de gradient classique peut être lente, bloquée, ou osciller.
D'où le besoin de techniques avancées.

Problème avec SGD

SGD hésite dans les zones à faible pente : $\nabla \approx 0 \rightarrow$ pas de mouvement.

Solution : Momentum

Idée : Accumuler de la vitesse comme une balle qui roule.

- Garde une fraction du gradient précédent
- Continue même si ∇ est petit

Analogie de la Balle

Comme une balle qui roule : elle garde sa vitesse et traverse les zones plates !

Algorithme

$$\begin{aligned}v_t &= \beta v_{t-1} + \alpha \nabla \mathcal{L}(w_t) \\w_{t+1} &= w_t - v_t\end{aligned}$$

Paramètres :

- α : learning rate (ex : 0.01)
- β : coefficient de momentum $\in [0, 1]$ (ex : 0.9)
- v_t : vitesse (velocity) à l'itération t

Interprétation :

- v_t = moyenne pondérée des gradients précédents
- β contrôle la mémoire (0.9 = garde 90% de la vitesse précédente)

Momentum : Avantages et Effet de β

Avantages :

- Gagne de la vitesse dans les descentes
- Ignore le bruit à court terme
- Traverse les plateaux efficacement

Effet de β

- $\beta = 0 \rightarrow$ GD classique (pas de mémoire)
- $\beta \approx 0.9 \rightarrow$ Bon compromis (mémoire + réactivité)
- $\beta \approx 0.99 \rightarrow$ Très longue mémoire (peut être trop lent)

Observation

Certains paramètres oscillent violemment, d'autres avancent trop lentement.

Exemple :

- Paramètre w_1 : gradients grands $\rightarrow$ oscille
- Paramètre w_2 : gradients petits $\rightarrow$ stagne
- Même learning rate pour les deux $\rightarrow$ problème !

Solution

Adapter le learning rate **individuellement** pour chaque paramètre selon son historique.

- Si gradients grands $\rightarrow$ réduire α
- Si gradients petits $\rightarrow$ augmenter α
- Adaptation automatique !

Formules

$$s_t = \rho s_{t-1} + (1 - \rho)(\nabla \mathcal{L})^2$$
$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{s_t} + \epsilon} \nabla \mathcal{L}$$

Paramètres :

- α : learning rate de base
- ρ : facteur de décroissance (ex : 0.9)
- ϵ : petit nombre (ex : 10^{-8}) pour éviter division par 0
- s_t : moyenne mobile des gradients au carré

Interprétation :

- $s_t \approx$ variance des gradients récents
- $\frac{\alpha}{\sqrt{s_t}}$ = learning rate adaptatif
- Grand $s_t \rightarrow$ petit learning rate

Résultat :

- "Calme" les dimensions qui oscillent
- "Réveille" les dimensions qui stagnent
- Convergence plus stable

Adam (Adaptive Moment Estimation)

Combine le meilleur de **Momentum** et de **RMSprop**.

De Momentum :

- Accumule la vitesse
- Traverse les plateaux
- Ignore le bruit

De RMSprop :

- Learning rate adaptatif
- Différent pour chaque paramètre
- Calme les oscillations

Pourquoi Adam ?

- Gère les gradients bruyants
- S'adapte à chaque paramètre
- Convergence rapide
- Standard de l'industrie (90% des cas)

Algorithme Complet

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L} \quad (\text{momentum})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L})^2 \quad (\text{RMSprop})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{correction de biais})$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Régularisation : Lutter contre l'Overfitting

Empêcher le réseau de "mémoriser" les données.

Le Fléau des Réseaux Profonds : L'Overfitting

- Un réseau de neurones a des millions de paramètres.
- Il est si puissant qu'il peut apprendre par cœur le bruit du dataset.
- **Symptôme** : Performance excellente en entraînement, médiocre en test.

Idée

À chaque étape d'entraînement, on "éteint" aléatoirement un pourcentage de neurones.

Pourquoi ça marche ?

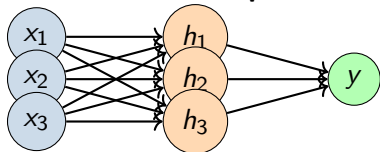
- Le réseau ne peut plus compter sur un seul neurone spécifique
- Il force la **redondance**
- Empêche la co-adaptation des neurones

Attention

On n'utilise pas le dropout pendant le test (on multiplie les poids par la probabilité d'activation).

Dropout : Visualisation

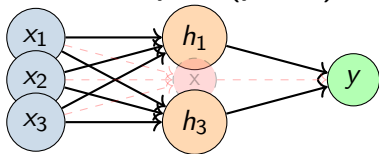
Sans Dropout :



Tous

les neurones actifs

Avec Dropout ($p=0.5$) :



Neurone h_2 désactivé

Effet

Le réseau apprend des représentations plus robustes car il ne peut pas dépendre d'un neurone spécifique.

Early Stopping

L'idée la plus simple

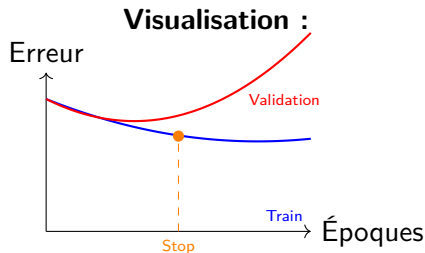
S'arrêter quand ça commence à mal tourner.

Principe :

- On surveille l'erreur sur un jeu de **validation**
- Dès que l'erreur de validation remonte, on s'arrête
- On garde les poids du meilleur modèle

Avantage

Empêche l'overfitting en arrêtant l'entraînement au bon moment.



Stabilisation : Batch Normalization

Normaliser au cœur du réseau pour stabiliser l'entraînement.

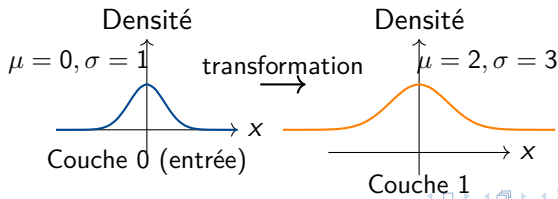
Le Problème : Internal Covariate Shift

Observation

On normalise les entrées X , mais après chaque couche, la distribution change !

Exemple concret :

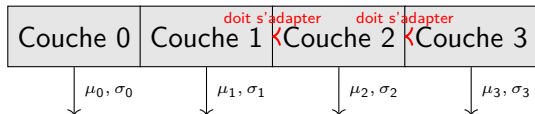
- Entrée normalisée : $\mu = 0, \sigma = 1$
- Après couche 1 : $\mu = 2, \sigma = 3$ (changé !)
- Après couche 2 : $\mu = -1, \sigma = 5$ (changé encore !)



Conséquence : Adaptation Constante

Le Problème

Les couches profondes doivent constamment s'adapter aux changements des couches précédentes.



Impact :

- Apprentissage plus lent
- Besoin de learning rates plus petits
- Sensibilité à l'initialisation

Solution :

- Normaliser après chaque couche
- Garder $\mu = 0, \sigma = 1$ constant
- **Batch Normalization**

La Solution : Batch Normalization

Idée : Normaliser après chaque couche pour garder $\mu = 0$ et $\sigma = 1$.

Algorithme (pendant l'entraînement)

Pour chaque mini-batch de taille m :

- 1 Calculer la moyenne : $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$
- 2 Calculer la variance : $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$
- 3 Normaliser : $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
- 4 Transformation apprise : $y_i = \gamma \hat{x}_i + \beta$

Pourquoi ?

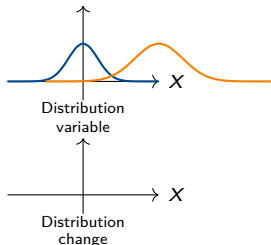
Paramètres appris :

- γ : scale (échelle)
- β : shift (décalage)

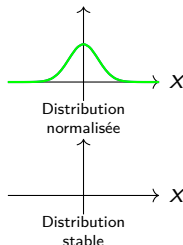
- Permet au réseau de "désactiver" la normalisation si nécessaire
- Si $\gamma = \sigma_B$ et $\beta = \mu_B$: pas de changement

Batch Normalization : Visualisation

Avant Batch Norm :



Après Batch Norm :



Résultat

Les distributions restent stables ($\mu = 0, \sigma = 1$) à travers toutes les couches.

Conclusion

- ① **Activations** : ReLU dans les couches cachées.
- ② **Initialisation** : He Initialization.
- ③ **Normalisation** : Batch Norm après chaque activation.
- ④ **Optimiseur** : Adam ($LR = 0.001$).
- ⑤ **Régularisation** : Dropout (0.2 - 0.5) + Early Stopping.

Conclusion

- Le Deep Learning est puissant mais nécessite de la discipline.
- Chaque brique (Activation, Init, Optim) a un rôle crucial.
- Le secret réside souvent dans la **stabilisation** du gradient.

Questions ?